

Reviewer Pack — Coxeter-Dynkin Diagrams and Circuit Complexity

Entry de21bcf95888 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 07:46:50 UTC

Coxeter-Dynkin Diagrams and Circuit Complexity

Entry ID: de21bcf95888 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 07:46:50 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Representation Theory (Coxeter Groups) **Field B** (complexity object): Complexity Theory: Boolean Circuit Satisfiability

Statement:

{‘text’: ‘For any given instance of a Boolean circuit C with n inputs, the number of distinct vertices in the associated Coxeter-Dynkin diagram for its symmetry group G is bounded by n^2 .’, ‘quantitative_relation’: ‘ $E[|V(G)|] = O(n^2)$ ’, ‘counterexample’: ‘An instance of a Boolean circuit C with symmetry group G having more than n^2 vertices’}}

Rationale (proposer’s reasoning):

{‘text’: ‘The Coxeter-Dynkin diagram captures the symmetries of the Boolean circuit, and its structure is expected to be influenced by the complexity of the circuit. A direct relationship between the number of vertices in the diagram and the circuit size could provide insights into circuit complexity.’, ‘influence_structure’: ‘The symmetry group of a circuit might be complex if the circuit itself is complex, suggesting that the number of vertices could serve as an indicator.’}}

Taxonomy category: COXETER_DYMKIN_DIAGRAMS (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 86368a6167a9e0a1

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a Boolean circuit C with n inputs, if the expected number of vertices in its associated Coxeter-Dynkin diagram G is $E[|V(G)|] \leq 1.5 * n^2$ for at least 25 out of 30 random seeds, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_circuit(n):
        # Generate a random Boolean circuit with n inputs
        return [random.choice([0, 1]) for _ in range(2**n)]

    def compute_symmetry_group(circuit):
        # Compute the symmetry group of the circuit (simplified version)
        # This is a placeholder function; actual implementation needed
        return set()

    def compute_coxeter_dynkin_diagram(group):
        # Compute the Coxeter-Dynkin diagram for the given group
        # This is a placeholder function; actual implementation needed
        return set()

    n_values = [5, 10, 15, 20, 30, 40]
    total_vertices = 0
    instances_tested = 0

    for n in n_values:
        for _ in range(5): # Test each n with 5 different circuits
            circuit = generate_circuit(n)
            group = compute_symmetry_group(circuit)
            diagram = compute_coxeter_dynkin_diagram(group)
            total_vertices += len(diagram)
            instances_tested += 1

    mean_vertices = total_vertices / instances_tested
    conjecture_holds = mean_vertices <= 1.5 * n**2
    for n in n_values:
        counterexample = "" if all(conjecture_holds) else "mapping_undefined"

    return {
```

```

        "metric_name": "mean_vertices",
        "metric_value": mean_vertices,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    if len(sys.argv) > 1:
        seeds = [int(s) for s in sys.argv[1:]]
    else:
        primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73,
        seeds = random.sample(primes, 30)

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_vertices = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if all(r["conjecture_holds"])) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_vertices} std=NA support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not all(r["conjecture_holds"]
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot verify the conjecture's support or falsification. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13289
2	preregistration	ollama_remote	glm4:latest	0	0	5876
3	novelty	ollama_remote	glm4:latest	0	0	4829
4	novelty	ollama_remote	glm4:latest	0	0	5054
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13027
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7385
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19193
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10315
9	judge	ollama_remote	glm4:latest	0	0	45318

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 124287 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/de21bcf95888.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/de21bcf95888.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/de21bcf95888.tar.gz` (if generated)